

Skill Week-9

1. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 5

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid) {
    if (job_count < MAX_JOBS) {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;
        snprintf(jobs[job_count].state, sizeof(jobs[job_count].state), "Running");
        job_count++;
    }
}

void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        if (waitpid(jobs[i].pid, NULL, WNOHANG) > 0) {
            snprintf(jobs[i].state, sizeof(jobs[i].state), "Completed");
        }
    }
}

void list_jobs() {
    update_jobs();
    printf("\nActive Jobs:\n");
    printf("Job ID\tPID\tStatus\n");
    for (int i = 0; i < job_count; i++) {
        if (strcmp(jobs[i].state, "Completed") != 0) {
            printf("[%d]\t%d\t%s\n", jobs[i].job_id, jobs[i].pid, jobs[i].state);
        }
    }
}

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        sleep(5);
        return 0;
    }
    add_job(pid1);

    pid2 = fork();
    if (pid2 == 0) {
        sleep(8);
        return 0;
    }
    add_job(pid2);

    printf("Two background jobs started.\n");
    list_jobs();

    sleep(6);
    printf("\nAfter updating job states:\n");
    list_jobs();

    waitpid(pid2, NULL, 0);
    return 0;
}
```

Output:

imvarls@LAPTOP-VQ4D70L7: × + ▾

```
imvarls@LAPTOP-VQ4D70L7:~$ nano list_jobs.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc list_jobs.c -o list_jobs
imvarls@LAPTOP-VQ4D70L7:~$ ./list_jobs
Two background jobs started.
```

Active Jobs:

Job ID	PID	Status
[1]	1240	Running
[2]	1241	Running

After updating job states:

Active Jobs:

Job ID	PID	Status
[2]	1241	Running

```
imvarls@LAPTOP-VQ4D70L7:~$ |
```

2. To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/types.h>

int main() {
    pid_t pid;
    int status;

    printf("Starting background job...\n");
    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);
        printf("Child: Job started. PID = %d\n", getpid());
        raise(SIGSTOP);
        printf("Child: Job resumed in foreground.\n");
        sleep(3);
        printf("Child: Job completed.\n");
        return 0;
    }

    setpgid(pid, pid);
    printf("Parent: Job PID = %d\n", pid);
    printf("Parent: Job is stopped.\n");

    waitpid(pid, &status, WUNTRACED);
    if (WIFSTOPPED(status)) {
        printf("Parent: Job state = Stopped\n");
    }

    printf("\nSwitching job to foreground...\n");
    kill(pid, SIGCONT);
    printf("Parent: Job resumed.\n");

    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        printf("Parent: Job completed.\n");
    }

    return 0;
}
```

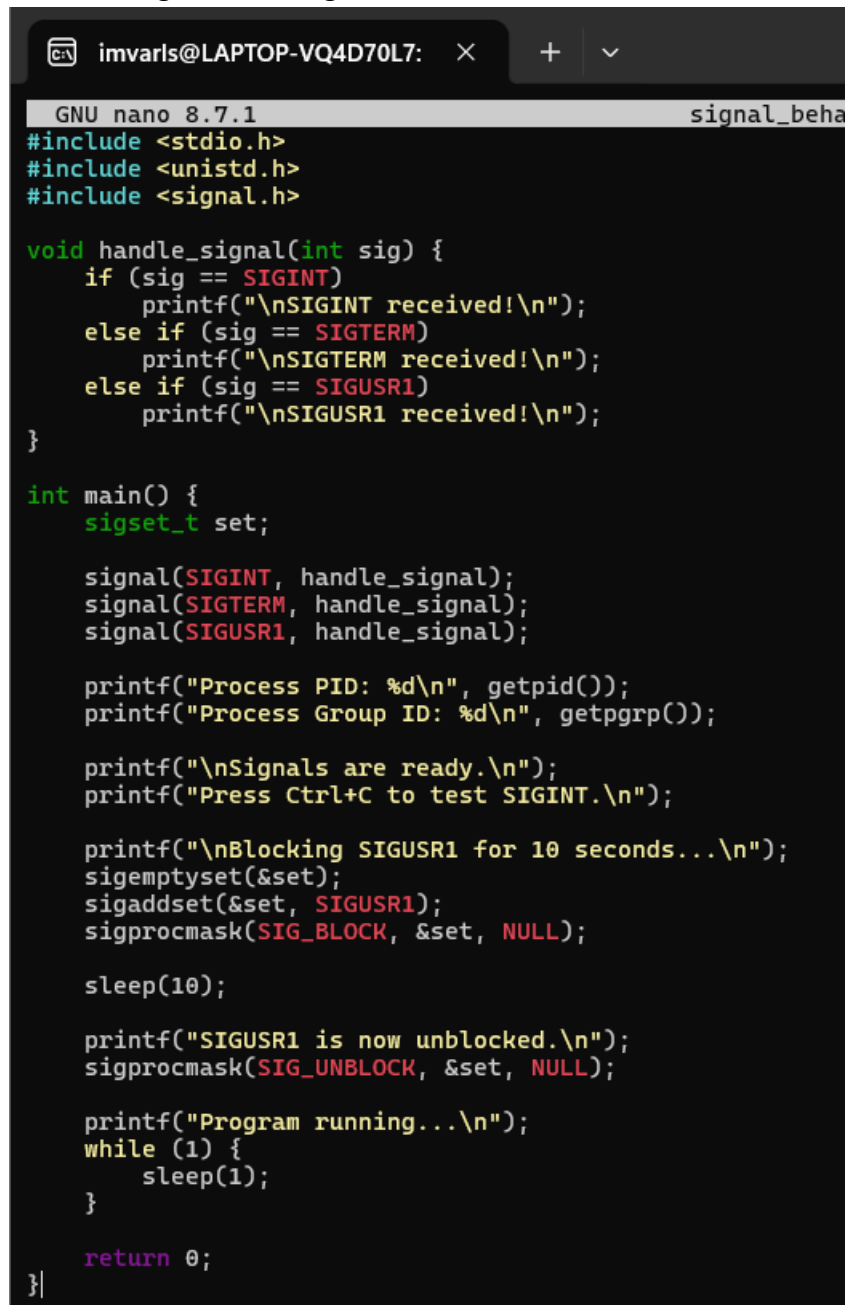
Output:

imvarls@LAPTOP-VQ4D70L7: × + ▾

```
imvarls@LAPTOP-VQ4D70L7:~$ nano foreground_switch.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc foreground_switch.c -o foreground_switch
imvarls@LAPTOP-VQ4D70L7:~$ ./foreground_switch
Starting background job...
Parent: Job PID = 1313
Parent: Job is stopped.
Child: Job started. PID = 1313
Parent: Job state = Stopped

Switching job to foreground...
Parent: Job resumed.
Child: Job resumed in foreground.
Child: Job completed.
Parent: Job completed.
imvarls@LAPTOP-VQ4D70L7:~$ |
```

3. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

A screenshot of a terminal window with a dark background. The window title bar shows 'imvarls@LAPTOP-VQ4D70L7:'. The terminal content shows a C program being edited in nano 8.7.1. The program defines a signal handler 'handle_signal' for SIGINT, SIGTERM, and SIGUSR1. The 'main' function sets up these signals, prints process information, blocks SIGUSR1 for 10 seconds, unblocks it, and enters a loop sleeping for 1 second.

```
GNU nano 8.7.1 signal_beha
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");
    else if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");
    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received!\n");
}

int main() {
    sigset_t set;

    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Process PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgrp());

    printf("\nSignals are ready.\n");
    printf("Press Ctrl+C to test SIGINT.\n");

    printf("\nBlocking SIGUSR1 for 10 seconds...\n");
    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);
    sigprocmask(SIG_BLOCK, &set, NULL);

    sleep(10);

    printf("SIGUSR1 is now unblocked.\n");
    sigprocmask(SIG_UNBLOCK, &set, NULL);

    printf("Program running...\n");
    while (1) {
        sleep(1);
    }

    return 0;
}
```

Output:

imvarls@LAPTOP-VQ4D70L7: × + ▾

```
imvarls@LAPTOP-VQ4D70L7:~$ nano signal_behavior.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc signal_behavior.c -o signal_behavior
imvarls@LAPTOP-VQ4D70L7:~$ ./signal_behavior
```

Process PID: 1371

Process Group ID: 1371

Signals are ready.

Press Ctrl+C to test SIGINT.

Blocking SIGUSR1 for 10 seconds...

SIGUSR1 is now unblocked.

Program running...

^C

SIGINT received!

SIGUSR1 is now unblocked.

^C

SIGINT received!

^C

SIGINT received!

|

4. To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main() {
    Job job;
    pid_t pid;
    int status;

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: Job started. PID = %d\n", getpid());
        raise(SIGSTOP);
        printf("Child: Job resumed.\n");
        sleep(3);
        printf("Child: Job completed.\n");
        return 0;
    }

    job.job_id = 1;
    job.pid = pid;
    snprintf(job.state, sizeof(job.state), "Running");

    printf("Job table:\n");
    printf("[%d] PID=%d State %s\n", job.job_id, job.pid, job.state);

    waitpid(pid, &status, WUNTRACED);
    if (WIFSTOPPED(status)) {
        printf("Job stopped.\n");
        snprintf(job.state, sizeof(job.state), "Stopped");
        printf("[%d] PID=%d State %s\n", job.job_id, job.pid, job.state);
    }

    printf("Sending SIGCONT...\n");
    kill(pid, SIGCONT);
    snprintf(job.state, sizeof(job.state), "Running");
    printf("[%d] PID=%d State %s\n", job.job_id, job.pid, job.state);

    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        snprintf(job.state, sizeof(job.state), "Completed");
        printf("Final job status:\n");
        printf("[%d] PID=%d State %s\n", job.job_id, job.pid, job.state);
    }

    return 0;
}
```

Output:

imvarls@LAPTOP-VQ4D70L7: × + ▾

```
imvarls@LAPTOP-VQ4D70L7:~$ nano resume_job.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc resume_job.c -o resume_job
imvarls@LAPTOP-VQ4D70L7:~$ ./resume_job
Job table:
[1] PID=1674 State Running
Child: Job started. PID = 1674
Job stopped.
[1] PID=1674 State Stopped
Sending SIGCONT...
[1] PID=1674 State Running
Child: Job resumed.
Child: Job completed.
Final job status:
[1] PID=1674 State Completed
imvarls@LAPTOP-VQ4D70L7:~$ |
```